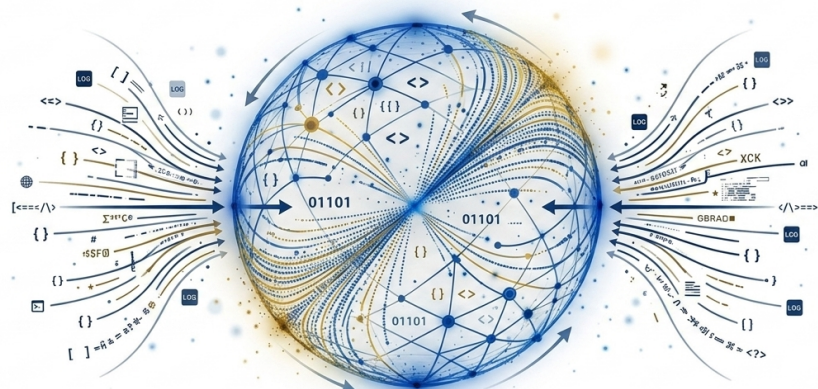




LOG SPHERE PROJECT

Capturing all logs of applications | Total Visibility

System Manual

One place to see everything your applications do: every request, every error, every user action.

Version 1.0 • July 2026

Contents

1. What is LogSphere?	3
2. How a log travels through the system	4
3. The building blocks	5
4. The dashboard, page by page	7
5. Alerts in plain words	9
6. Users, roles and permissions	10
7. How your data is protected	11
8. Connecting an application	12
9. Running the platform	13
10. Developer quick reference: saving logs from code	14
11. One-page memory aid	16
12. Appendix A: Complete event field reference	17

1. What is LogSphere?

LogSphere is a central home for the logs of all your software projects. Instead of every application writing its own log files that nobody reads, every application sends its logs to LogSphere. You then use one website (the dashboard) to see everything.

With LogSphere you can answer questions like:

- What happened during this request, step by step?
- Which errors are happening right now, and how often?
- Who changed this record, and what did it look like before?
- Which requests are slow?
- Is someone trying to break into my application?

The three promises

Promise	What it means
Never breaks your app	If LogSphere is down, your application keeps working. Logs are sent in the background and never block a user request.
Never stores secrets	Passwords, tokens, card numbers and similar values are removed or masked automatically before anything is saved.
Everyone sees only their own data	Each project is isolated. A user only sees the tenants and projects they were given access to.

Remember: One sentence to memorise: *applications send events to LogSphere; LogSphere cleans them, stores them, groups the errors, raises alerts, and shows everything in one dashboard.*

2. How a log travels through the system

Every log is called an **event**. This is the journey of one event, from your application to your screen:

1	Your application creates an event - for example: "user opened paper P-2026-01" or "payment failed".
2	The event is sent to LogSphere - over HTTP, together with the application's API key. Events are sent in small batches, in the background.
3	LogSphere checks the API key - the key tells LogSphere which tenant, project, application and environment the event belongs to. The sender cannot fake this.
4	The event is cleaned (sanitized) - sensitive fields like passwords or card numbers are removed or masked. Oversized content is safely cut.
5	The event goes into a durable queue - so nothing is lost even if a part of the system restarts.
6	A background worker stores it - into the PostgreSQL database, into monthly partitions that are easy to clean up later.
7	Errors are grouped - identical exceptions are recognised by a "fingerprint" and counted together as one exception group.
8	Alert rules are checked - every 30 seconds. If a rule matches (for example "more than 20 errors in 5 minutes"), an alert fires.
9	You see it in the dashboard - within about a second of it being sent.

Remember: If the log server is unreachable, the application keeps the events in a small memory buffer, retries a few times, and in the worst case drops the oldest logs. The application itself never crashes or slows down because of logging.

3. The building blocks

3.1 The hierarchy: who owns what

Everything in LogSphere lives inside a simple ladder. Learn it once and the whole admin area makes sense:

Level	What it is	Example
Tenant	A company or organisation. The top of the ladder.	Etea
Project	One product or system of that tenant.	MCQ
Application	One deployable piece of the project. Each application gets its own API key.	MCQ API, MCQ Web
Environment	Where it runs: Production, Staging, Development or Testing.	Testing
Module	A free label for the area inside the application.	SUBJECT, Auth, Web

3.2 The ten event types

Every event has exactly one type. The type decides which page shows it and how long it is kept.

Type	Use it for
ApiRequest	An HTTP request arrived.
ApiResponse	The request finished (with status code and duration).
Exception	An error happened. Shown on the Exceptions page, grouped by fingerprint.
Audit	A user changed business data. Old and new values are stored. Cannot be edited or deleted.
Performance	A measured operation (how long something took).
Integration	A call to an outside service (bank API, SMS gateway...).
BackgroundJob	A scheduled or queued job ran.
Security	Failed logins, invalid tokens, suspicious activity.
Custom	Your own business events, e.g. "PaperOpened".
Application	General log lines that fit nowhere else.

3.3 The six severity levels

From quietest to loudest: **Trace**, **Debug**, **Information**, **Warning**, **Error**, **Critical**. Errors and Criticals drive the error rate, the Exceptions page and most alerts.

3.4 The four IDs (this is the part worth memorising)

ID	Identifies	Memory aid
Event ID	One single log record.	one line
Correlation ID	All events of one request or workflow.	one story
Trace ID + Span ID	The same story in the standard W3C format; spans are the single steps inside it (a database call, an external call). Drawn as a waterfall in the Trace Viewer.	one story, drawn
Business Entity Type + ID	One business record across all requests and days, e.g. Subject SUB-001.	one life history

Remember: The correlation ID is the glue. Give every request one correlation ID, put it on every event of that request, and pass it on to services you call. One click on "View correlation" then shows the whole story.

4. The dashboard, page by page

4.1 Overview

The landing page. Big numbers for the chosen time range: total events, error rate, average and P95/P99 duration, requests per minute, active exception groups, queue depth and ingestion health. Below: events over time, the top failing routes, the most affected projects and the latest critical errors.

4.2 Log Explorer

The search page. Filter by project, application, environment, event types, severities, event ID, correlation ID, trace ID, user, business entity, HTTP route, status code or free text. Click a row to open the full event detail. From the detail you can copy the event ID, jump to the correlation (the whole story) or open the trace.

- **Live tail:** switches the list to real time. New events appear every few seconds. Pause any time.
- **Save search:** stores the current filters under a name; share or pin them on the Saved Searches page.
- **Export CSV:** downloads the filtered events (needs export permission; every export is recorded).

4.3 Traces

The Traces page lists the recent traces of the selected time range (with event count, errors and the longest step) and lets you open any trace by pasting its ID. Opening a trace shows the **waterfall**: every span is a bar, positioned by start time and sized by duration, indented under its parent. You see at a glance where the time went:

PAPER:GENERATE	=====	480 ms
- db: fn_paper_generate	=====	130 ms
- ext: sms-notify	. =====	160 ms

Traces in practice. A correlation ID already tells you WHAT happened during a request; spans additionally tell you WHERE THE TIME WENT. You only need them when you care about performance or when several services call each other. In .NET they are almost free - ASP.NET Core creates them for every request:

```
// on every event you send:
traceId = Activity.Current?.TraceId.ToString(),
spanId  = Activity.Current?.SpanId.ToString(),

// around a step you want to see as its own bar:
using var act = activitySource.StartActivity("db:fn_paper_generate");
// ... run the procedure ... then send one event with this span's
// spanId, parentSpanId and durationMs
```

Remember: For a packet-style API the highest-value setup is just two spans: root span = the packet, child span = the stored-procedure execution. That one split answers "is it the gateway or the database?" for every slow request.

4.4 Exceptions

Errors are not shown one by one but as **groups**: the same error, however often it happened, is one row with counters (total / last hour / last 24 h). Open a group to see:

- the occurrence trend chart and the sample stack trace,
- the **workflow panel**: status (New, Investigating, Identified, Resolved, Ignored), assignee, notes and a linked issue URL,
- the **activity trail**: who changed the status or assignment, and when,
- the **Copy AI prompt** button: copies a complete, ready-made prompt (context, stack trace, occurrences) that tells an AI assistant to investigate first and only then propose a fix.

Remember: If a Resolved error ever happens again, LogSphere automatically marks the group **Recurring**. That is how you catch regressions after a deployment.

4.5 Audit Explorer

Shows only Audit events: who did what to which business record. Each entry stores the old values and the new values side by side, the changed fields, the reason, the source IP and the correlation ID. Audit events cannot be changed or deleted, and are kept for years.

4.6 Alerts

Three tabs: **Occurrences** (alerts that fired; acknowledge and resolve them), **Rules** (what to watch for; see chapter 5) and **Channels** (where notifications go: email or a webhook such as Slack or Teams).

4.7 Saved Searches

Your named filters. Pinned ones appear first; shared ones are visible to your teammates. "Run" opens the Log Explorer with the filters applied.

4.8 Platform Health

The health of LogSphere itself: queue depth, dead letters (events that could not be processed), events per second, worker heartbeats and storage size per monthly partition. Green means healthy; this page auto-refreshes.

4.9 Admin

Management area with tabs for Tenants, Projects, Applications (and their API keys), Users, Redaction Rules, Retention Policies and the Access Audit (who logged in, viewed, exported or changed settings). You only see the tabs and rows your role allows.

5. Alerts in plain words

A rule is a question that LogSphere asks your logs every 30 seconds. Each rule has:

- **Condition type** - which question to ask (table below),
- **Threshold** - the number that triggers it (a count, a percent, or milliseconds),
- **Window** - how far back to look (e.g. the last 5 minutes),
- **Cooldown** - how long to stay silent after firing, so you are not spammed.

Condition type	The question it asks	Typical use
ErrorCountThreshold	Are there more than N errors in the window?	20 errors / 5 min: something is broken.
ErrorRatePercent	What percent of all events are errors?	5% / 10 min: better when traffic goes up and down.
SameFingerprintCount	Is one specific exception repeating?	Same error 100x / 10 min: a bad deployment.
DurationP95Threshold	Are requests getting slow?	P95 above 3000 ms: users are waiting.
SeverityDetected	Did anything at severity X or above happen at all?	Any Critical: zero-tolerance alarm.
AuthFailureCount	Too many security events?	30 / 5 min: possible brute-force attack.
IngestSilence	Has the application stopped logging completely?	Silence for 15 min: the app is probably down.
QueueDepth	Is LogSphere itself falling behind?	Queue above 50,000: platform alarm for admins.

A good starter set for any production project

- ErrorCountThreshold - 20 errors / 5 min / cooldown 15
- SeverityDetected (Critical) - window 5 min / cooldown 30
- DurationP95Threshold - 3000 ms / 10 min / cooldown 30
- IngestSilence - window 15 min / cooldown 60

6. Users, roles and permissions

Dashboard users log in with a username and password. What they can see and do is decided by their **grants**. A grant is: a role, plus the scope it applies to (tenant, project, environment), plus three extra switches.

Role	Meant for
SuperAdmin	The platform owner. Sees and manages everything, including global rules and all users.
TenantAdmin	Manages one tenant: its projects, applications, keys, users, rules and alerts.
ProjectAdmin	Manages one project and its alerts.
Developer / Support / Operations	Work with logs, traces and exceptions in their scope.
SecurityAuditor / ComplianceAuditor	Read security and audit data.
ReadOnly	May only look.

The three extra switches on every grant:

- **Can view bodies** - may see request/response payloads (otherwise they are hidden).
- **Can export** - may download CSV/JSON exports.
- **Can view security** - may see Security events.

Remember: Scoping is enforced by the server, not just hidden in the menu. A TenantAdmin who queries the API directly still only receives their own tenant's data, their own users, their own alert rules. Super-admin accounts are invisible to them.

7. How your data is protected

7.1 Secrets never reach the disk

Cleaning happens twice: once in the sending application (SDK) and once again on the server. The server never trusts the sender. Fields whose names contain words like *password*, *token*, *secret*, *api_key*, *authorization*, *cookie*, *cvv*, *card_number*, *account_number*, *iban*, *cnic*, *otp*, *biometric* are removed, replaced with [REDACTED], or masked to the last 4 characters. Administrators can add their own redaction rules per tenant or project; the global rules can never be switched off by a tenant.

7.2 Audit events are immutable

The database itself refuses UPDATE and DELETE on audit and security events. Nobody can quietly rewrite history - not even an administrator.

7.3 Retention: how long things are kept

Data	Default retention
Trace / Debug logs	7 days
Information logs	30 days
API requests / responses, performance	90 days
Exceptions	180 days
Security events	1 year
Audit events	~7 years

Old data is removed automatically by dropping whole monthly partitions - fast and clean. Policies can be changed per tenant, project, environment, event type or severity in Admin > Retention. A **legal hold** flag pauses deletion when data must be preserved.

7.4 Everything is watched, including the watchers

Logging into the dashboard, viewing an event, exporting data and every admin change is itself recorded in the Access Audit (Admin > Access Audit).

8. Connecting an application

Four steps, once per application:

1	Create the ladder - in Admin: Tenant, then Project, then Application. Codes are suggested automatically (lowercase-with-dashes).
2	Create an API key - Admin > Applications > Credentials > New. The key is shown ONCE - copy it immediately and store it in the application's configuration, never in source code.
3	Add the logging code - either the .NET SDK (one registration line plus <code>app.UseLogSphere()</code>) or plain HTTP POSTs to <code>/api/v1/events/batch</code> . The document <code>docs/INTEGRATION.md</code> (or the <code>nsd-log</code> skill) contains everything, ready to hand to a developer or an AI assistant.
4	Verify - send one test event, then check the Log Explorer. Force one error and check the Exceptions page.

The golden rules for every integration

- Send logs in the background, in batches. Never make a user wait for logging.
- One correlation ID per request, on every event, passed to downstream services.
- Old values in `requestData`, new values in `responseData` for audit events.
- Never send passwords, tokens or card data. Never send files or images.
- Set `businessEntityType` / `businessEntityId` so records have a searchable life history.

The one endpoint to remember

```
POST /api/v1/events/batch
Header: X-LogSphere-Key: ls_XXXXXXXX.XXXXXXXXX...
Body: { "events": [ { "eventType": "...", ... } ] }
```

9. Running the platform

Piece	What it does
logsphere-web (port 8080)	Nginx. Serves the dashboard and forwards /api to the API.
logsphere-api	The API plus three background workers: persistence (stores events), alerts (checks rules), retention (creates and drops partitions).
PostgreSQL	The database. All events, settings and users live here.

Everything runs in Docker and restarts automatically with the machine (*docker compose up -d* in the deploy folder). Configuration and secrets live in *deploy/.env*. Database schema changes apply themselves at API start-up.

Deploying through UAT and production

Schema changes travel with the code as numbered files in *db/migrations*. Each environment tracks what it has applied, so promotion is simply: deploy to UAT (its database updates itself, test there), then deploy the same build to production (the identical files apply there). The schema can never drift from the code version that is running. A lock ensures that when several API instances start together, only one migrates.

- Back up the production database before every deploy: *pg_dump -F c -f backup.dump logsphere*.
- Never edit an already-applied migration file - always add a new numbered file.
- Keep changes additive (new tables/columns/indexes) so the previous version could still run.
- Organisations that require a manual approval gate set *Database:AutoMigrate=false* in production and run migrations as an explicit step: *dotnet LogSphere.Api.dll --migrate-only*.
- Health check for machines: *GET /health* and *GET /health/ready*.
- Health check for humans: the Platform Health page.
- Back up the PostgreSQL database regularly - it is the single source of truth.

Useful commands (from the deploy folder): *docker compose stop / start*, *docker logs logsphere-api -f*, *docker compose up -d --build* after code changes.

10. Developer quick reference: saving logs from code

Replace two values in every example: **BASE_URL** (e.g. `http://localhost:8080`) and **API_KEY** (create it in Admin > Applications > Credentials - it is shown once). There is only one endpoint, and every kind of log goes through it:

The one endpoint

```
POST BASE_URL/api/v1/events/batch
Header: X-LogSphere-Key: API_KEY
Body:   { "events": [ { ...one or more events... } ] }
Answer: HTTP 202 { "accepted": 1 }
```

10.1 The smallest useful event

```
{
  "eventType": "Application",
  "severity": "Information",
  "message": "hello from my app",
  "module": "Setup"
}
```

Add more fields when useful: *correlationId* (same value on every event of one request), *userId*, *businessEntityType/Id*, *http*, *durationMs*, *exception*, *properties*.

10.2 C# / ASP.NET Core (with the LogSphere SDK)

```
// Program.cs - one-time setup: requests, responses and
// exceptions are then logged automatically
builder.Services.AddCentralLogging(o => {
    o.Endpoint = "BASE_URL"; o.ProjectKey = "API_KEY";
    o.ApplicationName = "Billing API"; o.Environment = "Production";
});
app.UseLogSphere();
```

```
// when business data changes (audit) and for domain events:
await auditLogger.LogAsync("InvoiceApproved", "Invoice", id,
    oldValues: before, newValues: after);
activityLogger.LogEvent("PaymentReconciled", new { amount },
    entityType: "Payment", entityId: payId);
```

10.3 JavaScript / Node.js

```

async function logEvent(e) {
  try {
    await fetch(process.env.LOGSPHERE_URL + "/api/v1/events/batch", {
      method: "POST",
      headers: { "Content-Type": "application/json",
        "X-LogSphere-Key": process.env.LOGSPHERE_KEY },
      body: JSON.stringify({ events: [e] }) });
  } catch { /* logging must never crash the app */ }
}

logEvent({ eventType: "Custom", actionName: "PaperOpened",
  userId: "1001", module: "Exams",
  businessEntityType: "Paper", businessEntityId: "P-01" });

```

10.4 Web pages (browser applications)

Never put the API key inside a web page - anyone can read it there. A browser app sends its logs to **its own backend**, and the backend forwards them to LogSphere with the key:

```

// browser:      fetch("/my-api/log", { method: "POST",
//               body: JSON.stringify(event) })
// your backend: receives it, adds the API key,
//               forwards to BASE_URL/api/v1/events/batch

```

10.5 Python

```

import requests

def log_event(e):
    try:
        requests.post(BASE_URL + "/api/v1/events/batch",
            json={"events": [e]},
            headers={"X-LogSphere-Key": API_KEY}, timeout=5)
    except Exception:
        pass # logging must never break the app

```

10.6 Audit events: one rule to remember

```

{ "eventType": "Audit", "actionName": "SubjectUpdated",
  "userId": "1001", "businessEntityType": "Subject",
  "businessEntityId": "SUB-001",
  "requestData": { "status": "Draft" }, // OLD values
  "responseData": { "status": "Approved" } } // NEW values

```

Remember: Old values go in *requestData*, new values in *responseData*. That is what draws the before/after comparison in the Audit Explorer.

For production use, send events in background batches with retries instead of one by one - ready-made sender code for every language is in *docs/INTEGRATION.md*, written so it can also be handed to an AI assistant to do the whole integration.

11. One-page memory aid

Question	Answer
What is LogSphere in one line?	Apps send events; LogSphere cleans, stores, groups, alerts and shows them.
What is an event?	One log record with a type (1 of 10) and a severity (1 of 6).
The ladder?	Tenant > Project > Application > Environment > Module.
The four IDs?	Event = one line. Correlation = one story. Trace/Span = the story drawn. Entity = one record's life.
Where do I search logs?	Log Explorer (live tail for real time).
Where do I see errors?	Exceptions - grouped, with workflow, activity trail and the AI prompt button.
Who changed this record?	Audit Explorer - old vs new values, immutable.
How do alerts work?	A question + threshold + window + cooldown, checked every 30 s.
Is the platform itself OK?	Platform Health page.
How do I connect an app?	Create app in Admin, copy the API key once, POST events to /api/v1/events/batch.
What is never stored?	Passwords, tokens, card data, binary files. Cleaned twice, client and server.
Who sees what?	Only what their grants allow - enforced by the server.

Dashboard address: <http://localhost:8080> (or your server's address). Sign in, open the Overview, and start from there.

Appendix A. Complete event field reference

Section 10 showed the smallest event. This appendix lists **every field an application can send when saving a log**, what it is for, and whether it is required. The rule is simple: only **eventType** is mandatory - everything else is optional and has a sensible default. The four identity fields (tenant, project, application, environment) are **never sent**; the server derives all four from the API key and ignores any values in the body.

A.1 What is actually required

Saving a log needs just two inputs: the API key **header**, and **eventType** in the body. The smallest possible accepted request is:

```
POST BASE_URL/api/v1/events (or /events/batch)
Header: X-LogSphere-Key: API_KEY
Body: { "eventType": "Custom" }
```

An event is only **rejected** for one of these reasons:

Rejected when...	Meaning
empty / unparseable	body is not a JSON object, or is null
> 256 KB	a single event exceeds the size cap
unknown eventType	missing, or not one of the 10 valid types (see A.9)
unknown severity	a severity was sent but is not a valid name
unknown status	a status was sent but is not a valid value
unsupported schemaVersion	schemaVersion does not start with "1."

If none of those apply, the event is accepted (HTTP 202) and processed durably in the background. When you use the .NET SDK, it fills eventId, eventTimestamp, severity, schemaVersion, correlation/trace ids, user, machine and version fields for you - so in practice you supply even less.

A.2 Core fields

Field	Req.	Default	Purpose
eventType	Yes	-	The category of the record. One of the 10 types in A.9.
severity	No	Information	Trace / Debug / Information / Warning / Error / Critical.
status	No	(none)	Operation state, e.g. Completed, Failed (see A.9).
message	No	(none)	Human-readable text. Max 4000 characters.
eventId	No	auto GUID	Client-supplied id; enables idempotent retries.
eventTimestamp	No	now	When it happened. Clamped to now if >1h future or >30d past.
schemaVersion	No	1.0	Envelope version; must be 1.x.

A.3 Correlation & tracing

Field	Purpose
correlationId	Ties all events of one logical request/flow into one story. Put the same value on every event. Max 128.
traceId / spanId / parentSpanId	Distributed-trace linkage that draws the trace tree (128 / 64 / 64).
sequence	Orders events within a single trace.

A.4 Who and what - business context

Field	Purpose
module / component	Where in the application the event came from (256 / 256).
actionName	Operation name, e.g. SAVE_ROLE, OrderUpdated (256).
userId / userName / sessionId	Who did it - drives the Actor column (256 each).
businessEntityType / businessEntityId	What record was acted on - drives the Entity column and a record's searchable life history (128 / 256).

A.5 HTTP details (http: { ... })

Field	Purpose
method / route / statusCode	Request line and result. A query string in route is sanitized automatically.
clientId	Caller IP address - drives the audit Source IP field (64).
userAgent / requestSize / responseSize	Client agent and payload sizes.

A.6 Timing

Field	Purpose
durationMs	Total operation time (milliseconds) - the basis of Performance events.
dbDurationMs / externalDurationMs	Time spent in the database / in downstream calls.

A.7 Deployment metadata

Field	Purpose
machineName	Host that produced the event (SDK fills this).
applicationVersion / deploymentVersion	Build and release provenance (SDK fills these from options).

A.8 Payload objects (cleaned and size-capped on the server)

Field	Purpose
requestData	Request body. For Audit events this holds the OLD values.
responseData	Response body. For Audit events this holds the NEW values.
properties	Any structured data you want. Audit reason and changedFields live here - reason drives the audit Reason field.
exception	type, message, stackTrace, errorCode, className, methodName, fileName, lineNumber, innerExceptions[] - used to group and fingerprint exceptions.

A.9 Valid values and limits

Setting	Allowed values / limit
eventType (10)	ApiRequest, ApiResponse, Audit, Exception, Performance, Integration, BackgroundJob, Security, Custom, Application
severity (6)	Trace, Debug, Information, Warning, Error, Critical
status (8)	Started, InProgress, Completed, Failed, Retrying, Cancelled, TimedOut, PartiallyCompleted
Size caps	256 KB per event • 500 events per batch • message 4000 chars

A.10 Filling the Source IP and Reason fields

If an audit record shows a blank **Source IP** or **Reason**, it is because those two optional inputs were not sent - nothing on the LogSphere side needs changing. Populate them from the sending application:

Audit field	Comes from the input
Source IP	<code>http.clientIp</code>
Reason	<code>properties.reason</code> (the SDK LogAsync "reason:" argument)

Everything else in the Audit Explorer - When, Actor, Entity, Action, and the before/after value grid - is filled from the core, context and payload fields above.